

Big O Notation: Complete Guide to Algorithm Analysis

Introduction to Algorithm Analysis

In computer science, **algorithm analysis** is the process of determining the computational complexity of algorithms—the amount of time, storage, or other resources needed to execute them^{[186][191]}. Rather than measuring actual execution time (which varies by hardware, compiler, and system load), we use **asymptotic analysis** to express how resource requirements grow as input size increases^{[185][188][193]}.

This mathematical approach allows us to compare algorithms independently of implementation details, focusing on fundamental efficiency characteristics that remain constant across different computing environments^{[186][191]}.

Why Algorithm Analysis Matters

Understanding algorithmic efficiency is critical for several reasons^{[185][199]}:

- 1. Predicting Performance:** Estimate how algorithms behave as data scales from hundreds to millions of records
- 2. Comparing Solutions:** Objectively evaluate different approaches to the same problem
- 3. Optimizing Code:** Identify bottlenecks and opportunities for improvement
- 4. Resource Planning:** Make informed decisions about memory, processing power, and infrastructure needs
- 5. Interview Success:** Essential knowledge for technical interviews at top companies
- 6. Problem-Solving Strategy:** Guide selection of appropriate algorithms and data structures

The Problem with Actual Time Measurement

Consider timing how long it takes to sort an array. The actual time depends on:

- CPU speed and architecture
- Programming language and compiler optimizations
- Operating system and background processes
- System memory and cache size
- Input data characteristics (already sorted, reverse sorted, random)

These factors make direct time comparisons meaningless. A slower algorithm on fast hardware might outperform a better algorithm on slow hardware for small inputs^{[186][193]}.

Asymptotic Notation: The Mathematical Foundation

Asymptotic notation provides a mathematical framework to describe algorithm behavior as input size approaches infinity^{[188][191][200]}. It focuses on the **growth rate** or **order of magnitude** rather than exact values.

The Three Primary Notations

Algorithm analysis uses three main asymptotic notations, each serving a distinct purpose^{[187][190][192]}:

1. Big O Notation (O) - Upper Bound

Definition: $O(f(n))$ represents the **worst-case** upper bound on algorithm growth^{[188][190][196]}.

Formal definition: $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that:

$$f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

Meaning: The algorithm will **never** perform worse than this bound for sufficiently large inputs^{[185][187]}.

Use Case: Most commonly used in practice because it guarantees maximum resource usage^{[188][193]}.

Example: If an algorithm has time complexity $O(n^2)$, it means for large n , the runtime will not exceed some constant times n^2 .

2. Big Omega Notation (Ω) - Lower Bound

Definition: $\Omega(f(n))$ represents the **best-case** lower bound^{[190][192][194]}.

Formal definition: $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that:

$$f(n) \geq c \cdot g(n) \text{ for all } n \geq n_0$$

Meaning: The algorithm will require **at least** this many resources in the best scenario^{[187][192]}.

Use Case: Less commonly used; useful for proving lower bounds on problem difficulty.

Example: Binary search is $\Omega(1)$ in the best case (target is at middle) and $O(\log n)$ in the worst case.

3. Big Theta Notation (Θ) - Tight Bound

Definition: $\Theta(f(n))$ represents a **tight bound** when upper and lower bounds match^{[189][190][192]}.

Formal definition: $f(n) = \Theta(g(n))$ if there exist positive constants c_1 , c_2 , and n_0 such that:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0$$

Meaning: The algorithm's growth rate is **exactly** this order, bounded both above and below^{[187][195]}.

Use Case: Most precise notation; used when an algorithm's performance is consistent across cases.

Example: Linear search is $\Theta(n)$ because it always examines elements sequentially regardless of case.

Comparison of Notations

Notation	Meaning	Analogy	When to Use
O (Big O)	Upper bound ($\leq$)	Maximum speed limit	Worst-case analysis (most common)
Ω (Omega)	Lower bound ($\geq$)	Minimum speed guarantee	Best-case analysis
Θ (Theta)	Tight bound ($=$)	Exact speed	Average or all cases identical

Common Time Complexities

Understanding standard complexity classes is essential for recognizing algorithm efficiency^{[50][193]}
[^199]:

1. O(1) - Constant Time

Characteristics:

- Runtime independent of input size
- Single operation or fixed number of operations
- Most efficient possible complexity

Examples^{[50][193]}:

- Array access by index: `arr[5]`
- Hash table lookup (average case)
- Arithmetic operations: `a + b`
- Variable assignment

Code Example:

```
int getFirstElement(int arr[], int size) {
    return arr[0]; // Always one operation
}
```

2. O(log n) - Logarithmic Time

Characteristics:

- Divides problem size at each step (typically by half)
- Extremely efficient for large datasets

- Common in divide-and-conquer algorithms

Examples^[50][207]:

- Binary search in sorted array
- Finding element in balanced binary search tree
- Some divide-and-conquer algorithms

Intuition: With each operation, the problem size shrinks dramatically. For $n=1,000,000$, only about 20 operations needed.

Code Example:

```
int binarySearch(int arr[], int size, int target) {
    int left = 0, right = size - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

3. $O(n)$ - Linear Time

Characteristics:

- Processes each input element once
- Runtime directly proportional to input size
- Very common and generally acceptable

Examples^[50][193]:

- Linear search in unsorted array
- Finding min/max in array
- Traversing linked list
- Single loop through data

Code Example:

```
int findMax(int arr[], int size) {
    int max = arr[0];
    for (int i = 1; i < size; i++) { // n iterations
        if (arr[i] > max) max = arr[i];
    }
    return max;
}
```

4. $O(n \log n)$ - Linearithmic Time

Characteristics:

- Optimal for comparison-based sorting
- Common in efficient divide-and-conquer algorithms
- Significantly better than $O(n^2)$

Examples^[50][207]:

- Merge sort, quicksort (average case), heapsort
- Efficient tree operations
- Fast Fourier Transform

Intuition: Divides problem ($\log n$) and processes all elements (n) at each level.

5. $O(n^2)$ - Quadratic Time

Characteristics:

- Nested iterations over data
- Performance degrades rapidly with size
- Acceptable only for small datasets

Examples^[50][193]:

- Bubble sort, selection sort, insertion sort
- Naive string matching
- Checking all pairs in array

Code Example:

```
void bubbleSort(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {           // n iterations
        for (int j = 0; j < size - i - 1; j++) {   // n iterations
            if (arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```

6. $O(2^n)$ - Exponential Time

Characteristics:

- Doubles with each additional input element
- Impractical for $n > 30$ approximately
- Often indicates brute-force approach

Examples^[193][204]:

- Recursive Fibonacci (naive implementation)
- Generating all subsets
- Tower of Hanoi
- Traveling salesman (brute force)

Code Example:

```
int fibonacci(int n) {  
    if (n <= 1) return n;  
    return fibonacci(n-1) + fibonacci(n-2); // Two recursive calls  
}
```

7. $O(n!)$ - Factorial Time

Characteristics:

- Worst possible practical complexity
- Generates all permutations
- Unusable for $n > 10-12$

Examples:

- Brute force traveling salesman
- Generating all permutations
- Some backtracking algorithms

Complexity Hierarchy (Best to Worst)

$O(1) \ll O(\log n) \ll O(n) \ll O(n \log n) \ll O(n^2) \ll O(n^3) \ll O(2^n) \ll O(n!)$

Performance Comparison (operations for different input sizes)^[50][193]:

Complexity	n=10	n=100	n=1000	n=1,000,000
$O(1)$	1	1	1	1
$O(\log n)$	3	7	10	20

Complexity	n=10	n=100	n=1000	n=1,000,000
$O(n)$	10	100	1,000	1,000,000
$O(n \log n)$	30	700	10,000	20,000,000
$O(n^2)$	100	10,000	1,000,000	10^{12}
$O(2^n)$	1,024	10^{30}	10^{300}	Impossible

Rules for Calculating Big O

Rule 1: Drop Constants

Constants don't affect growth rate^{[193][196][^199]}.

Examples:

- $O(2n) \rightarrow O(n)$
- $O(500) \rightarrow O(1)$
- $O(n/2) \rightarrow O(n)$
- $O(13n^2) \rightarrow O(n^2)$

Reasoning: For large n , the constant becomes insignificant. An algorithm doing $2n$ operations vs. n operations both scale linearly.

Rule 2: Drop Non-Dominant Terms

Keep only the fastest-growing term^{[193][196][^199]}.

Examples:

- $O(n^2 + n) \rightarrow O(n^2)$
- $O(n + \log n) \rightarrow O(n)$
- $O(n^3 + n^2 + n) \rightarrow O(n^3)$
- $O(2^n + n^2) \rightarrow O(2^n)$

Reasoning: As n grows large, the dominant term overwhelms others. For $n=1000$, $n^2 = 1,000,000$ while $n = 1,000$ (negligible difference).

Rule 3: Different Inputs = Different Variables

Use separate variables for independent inputs^{[193][199]}.

Example:

```
void process(int arrA[], int sizeA, int arrB[], int sizeB) {
    for (int i = 0; i < sizeA; i++) { } // O(a)
    for (int j = 0; j < sizeB; j++) { } // O(b)
```

```

}
// Total:  $O(a + b)$ , NOT  $O(n)$ 

```

Rule 4: Sequential Operations Add

Consecutive loops/operations sum their complexities^{[193][199]}.

```

void example(int arr[], int size) {
    for (int i = 0; i < size; i++) { }    //  $O(n)$ 
    for (int i = 0; i < size; i++) { }    //  $O(n)$ 
    for (int i = 0; i < size; i++) { }    //  $O(n)$ 
}
// Total:  $O(n) + O(n) + O(n) = O(3n) = O(n)$ 

```

Rule 5: Nested Operations Multiply

Nested loops multiply their complexities^{[193][199]}.

```

void example(int arr[], int size) {
    for (int i = 0; i < size; i++) {      // n times
        for (int j = 0; j < size; j++) {  // n times each
            printf("%d ", arr[i] + arr[j]);
        }
    }
}
// Total:  $O(n \times n) = O(n^2)$ 

```

Space Complexity

Space complexity measures memory usage relative to input size^{[194][208][220][223]}.

Components of Space Complexity

Total Space = Input Space + Auxiliary Space^{[205][211][220]}

Input Space: Memory for input data (usually can't be optimized)

Auxiliary Space: Extra temporary memory algorithm uses during execution^{[205][211][214]}

Example:

```

void reverseArray(int arr[], int size) {
    int temp[size]; // Auxiliary space:  $O(n)$ 
    for (int i = 0; i < size; i++) {
        temp[i] = arr[size - 1 - i];
    }
    // Space complexity:  $O(n)$  input +  $O(n)$  auxiliary =  $O(n)$ 
    // Auxiliary space:  $O(n)$ 
}

```



```
void reverseInPlace(int arr[], int size) {
    for (int i = 0; i < size/2; i++) {
        int temp = arr[i]; // Auxiliary space: O(1)
        arr[i] = arr[size - 1 - i];
        arr[size - 1 - i] = temp;
    }
    // Space complexity: O(n) input + O(1) auxiliary = O(n)
    // Auxiliary space: O(1)
}
```

Recursion and Space Complexity

Recursive calls use stack space^{[208][220][^223]}!

Example:

```
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
// Space complexity: O(n) due to n recursive calls on stack
```

Call Stack for factorial(5):

```
factorial(5)
  → factorial(4)
    → factorial(3)
      → factorial(2)
        → factorial(1)
```

Each call occupies stack memory until it returns.

Common Space Complexities

Space	Example
O(1)	Iteration with fixed variables
O(log n)	Balanced tree recursion
O(n)	Additional array, linear recursion
O(n ²)	2D matrix

Analyzing Recursive Algorithms

Recursive algorithm analysis uses **recurrence relations**^{[204][207][210][213]}.

General Approach

1. Express runtime as recurrence: $T(n)$ in terms of $T(\text{smaller input})$
2. Identify base case
3. Solve recurrence to get closed form

Example 1: Linear Recursion

```
int sum(int n) {  
    if (n == 1) return 1;  
    return n + sum(n - 1);  
}
```

Recurrence^[210]_[216]:

- $T(1) = 1$ (base case)
- $T(n) = 1 + T(n-1)$ (one operation + recursive call)

Solution:

```
T(n) = 1 + T(n-1)  
      = 1 + (1 + T(n-2))  
      = 2 + T(n-2)  
      = k + T(n-k)  
      = (n-1) + T(1)  
      = n
```

Time Complexity: $O(n)$

Example 2: Binary Recursion

```
int fibonacci(int n) {  
    if (n <= 1) return n;  
    return fibonacci(n-1) + fibonacci(n-2);  
}
```

Recurrence^[204]_[207]:

- $T(0) = T(1) = 1$
- $T(n) = T(n-1) + T(n-2) + 1$

Solution: Forms binary tree with $\sim 2^n$ nodes

Time Complexity: $O(2^n)$

Example 3: Divide and Conquer

```
int binarySearch(int arr[], int left, int right, int target) {
    if (left > right) return -1;
    int mid = left + (right - left) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] > target)
        return binarySearch(arr, left, mid - 1, target);
    return binarySearch(arr, mid + 1, right, target);
}
```

Recurrence^{[207][213]}:

- $T(1) = 1$
- $T(n) = T(n/2) + 1$

Solution: Halves problem each time

Time Complexity: $O(\log n)$

Amortized Analysis

Amortized analysis calculates average cost per operation over a sequence, accounting for occasional expensive operations^{[206][212][^215]}.

Dynamic Array Example

Consider appending to dynamic array that doubles when full^{[206][209][^215]}:

```
void append(DynamicArray* arr, int value) {
    if (arr->size == arr->capacity) {
        // Double capacity and copy all elements
        int* new_arr = malloc(2 * arr->capacity * sizeof(int));
        for (int i = 0; i < arr->size; i++) {
            new_arr[i] = arr->data[i];
        }
        free(arr->data);
        arr->data = new_arr;
        arr->capacity *= 2;
    }
    arr->data[arr->size++] = value;
}
```

Analysis^{[212][215][^218]}:

- Most appends: $O(1)$ (just insert)
- Occasional resize: $O(n)$ (copy all elements)
- Resize happens at sizes: 1, 2, 4, 8, 16, ..., 2^k

Total cost for n appends:

- Regular insertions: n operations

- Resizing copies: $1 + 2 + 4 + \dots + n = 2n$ operations
- **Total: $3n$ operations**

Amortized cost: $3n / n = O(1)$ per operation

Despite $O(n)$ worst-case, average is $O(1)$!

Implementation Examples in C

Example 1: Analyzing Different Loop Patterns

```
#include <stdio.h>
#include <time.h>

// O(1) - Constant time
int constantTime(int arr[], int size) {
    return arr[0] + arr[size-1]; // Fixed operations
}

// O(n) - Linear time
int linearTime(int arr[], int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) { // n iterations
        sum += arr[i];
    }
    return sum;
}

// O(n²) - Quadratic time
void quadraticTime(int arr[], int size) {
    for (int i = 0; i < size; i++) { // n iterations
        for (int j = 0; j < size; j++) { // n iterations
            printf("%d,%d ", arr[i], arr[j]);
        }
    }
}

// O(log n) - Logarithmic time
int logarithmicTime(int arr[], int size, int target) {
    int left = 0, right = size - 1;
    while (left <= right) { // Halves search space each time
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

// O(n log n) - Merge sort
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
```

```

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }

    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    int arr[] = {5, 2, 8, 1, 9, 3, 7, 4, 6};
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("=== Complexity Analysis Demonstration ===\n\n");

    printf("O(1) - Constant Time:\n");
    printf("First + Last = %d\n\n", constantTime(arr, size));

    printf("O(n) - Linear Time:\n");
    printf("Sum = %d\n\n", linearTime(arr, size));

    printf("O(log n) - Logarithmic Time (Binary Search):\n");
    int sorted[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
    printf("Found 7 at index: %d\n\n", logarithmicTime(sorted, 9, 7));

    printf("O(n log n) - Linearithmic Time (Merge Sort):\n");
    mergeSort(arr, 0, size - 1);
    printf("Sorted: ");
    for (int i = 0; i < size; i++) printf("%d ", arr[i]);
    printf("\n");

    return 0;
}

```

Example 2: Space Complexity Analysis

```

#include <stdio.h>
#include <stdlib.h>

// O(1) auxiliary space - in-place reversal
void reverseInPlace(int arr[], int size) {

```

```

    int left = 0, right = size - 1;
    while (left < right) {
        int temp = arr[left]; // Only one extra variable
        arr[left] = arr[right];
        arr[right] = temp;
        left++;
        right--;
    }
}

// O(n) auxiliary space - using extra array
void reverseCopy(int arr[], int size) {
    int* temp = (int*)malloc(size * sizeof(int)); // n extra space
    for (int i = 0; i < size; i++) {
        temp[i] = arr[size - 1 - i];
    }
    for (int i = 0; i < size; i++) {
        arr[i] = temp[i];
    }
    free(temp);
}

// O(n) space due to recursion stack
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
    // Stack depth = n, each call uses space
}

// O(1) space - iterative factorial
int factorialIterative(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}

// O(n2) auxiliary space - 2D array
void create2DMatrix(int n) {
    int** matrix = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        matrix[i] = (int*)malloc(n * sizeof(int));
    }
    // Use matrix...

    // Cleanup
    for (int i = 0; i < n; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

int main() {
    printf("=== Space Complexity Examples ===\n\n");
}

```

```

int arr[] = {1, 2, 3, 4, 5};
int size = 5;

printf("Original: ");
for (int i = 0; i < size; i++) printf("%d ", arr[i]);
printf("\n");

printf("\n(1) Space - In-place reversal:\n");
reverseInPlace(arr, size);
for (int i = 0; i < size; i++) printf("%d ", arr[i]);
printf("\n");

printf("\nRecursive factorial(5) - O(n) space: %d\n", factorial(5));
printf("Iterative factorial(5) - O(1) space: %d\n", factorialIterative(5));

return 0;
}

```

Example 3: Recursive Complexity Analysis

```

#include <stdio.h>

// T(n) = T(n-1) + 1 → O(n) time, O(n) space
int linearRecursion(int n) {
    if (n <= 0) return 0;
    return 1 + linearRecursion(n - 1);
}

// T(n) = 2T(n-1) + 1 → O(2^n) time, O(n) space
int exponentialRecursion(int n) {
    if (n <= 0) return 1;
    return exponentialRecursion(n - 1) + exponentialRecursion(n - 1);
}

// T(n) = T(n/2) + 1 → O(log n) time, O(log n) space
int binaryRecursion(int n) {
    if (n <= 1) return 1;
    return 1 + binaryRecursion(n / 2);
}

// T(n) = 2T(n/2) + n → O(n log n) time, O(log n) space
int divideConquer(int arr[], int left, int right) {
    if (left >= right) return 0;

    int mid = left + (right - left) / 2;
    int leftOps = divideConquer(arr, left, mid);
    int rightOps = divideConquer(arr, mid + 1, right);

    return (right - left + 1) + leftOps + rightOps;
}

void printComplexity(const char* name, int n, int result) {
    printf("%s(%d) = %d\n", name, n, result);
}

```

```

int main() {
    printf("=== Recursive Complexity Patterns ===\n\n");

    printf("O(n) - Linear Recursion:\n");
    printComplexity("linearRecursion", 5, linearRecursion(5));
    printComplexity("linearRecursion", 10, linearRecursion(10));

    printf("\nO(log n) - Binary Recursion:\n");
    printComplexity("binaryRecursion", 8, binaryRecursion(8));
    printComplexity("binaryRecursion", 16, binaryRecursion(16));

    printf("\nO(2^n) - Exponential Recursion:\n");
    printf("Warning: Exponential growth!\n");
    printComplexity("exponentialRecursion", 5, exponentialRecursion(5));
    printComplexity("exponentialRecursion", 10, exponentialRecursion(10));

    printf("\nO(n log n) - Divide and Conquer:\n");
    int arr[] = {1, 2, 3, 4, 5, 6, 7, 8};
    printf("Operations: %d\n", divideConquer(arr, 0, 7));

    return 0;
}

```

Best Practices for Algorithm Analysis

1. **Always consider worst case** unless specifically analyzing best/average
2. **Focus on large inputs** (asymptotic behavior)
3. **Identify dominant operations** (comparisons, assignments, etc.)
4. **Count carefully** in nested structures
5. **Consider both time and space** complexity
6. **Use proper notation** (don't confuse O , Ω , Θ)
7. **Simplify expressions** (drop constants and lower terms)
8. **Test assumptions** with actual data when possible

Summary

Big O notation is the cornerstone of algorithm analysis, providing a standardized way to compare algorithm efficiency. Understanding time and space complexity allows you to:

- Choose appropriate algorithms for your constraints
- Optimize critical code sections
- Predict how code scales with data growth
- Communicate performance characteristics clearly

Master these concepts through practice—analyze every algorithm you encounter, and complexity analysis will become second nature.

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20] [21] [22] [23] [24] [25] [26] [27] [28] [29] [30] [31] [32] [33] [34] [35] [36] [37] [38] [39]

**

1. <https://www.simplilearn.com/big-o-notation-in-data-structure-article>
2. <https://www.datacamp.com/tutorial/big-o-notation-time-complexity>
3. <https://www.simplilearn.com/tutorials/data-structure-tutorial/time-and-space-complexity>
4. <https://www.cs.auckland.ac.nz/courses/compsci220s1c/lectures/2014S1C/Part1/220-03.pdf>
5. https://en.wikipedia.org/wiki/Big_O_notation
6. <https://edurev.in/t/83450/4--Asymptotic-worst-case-time-and-space-complexity>
7. https://www.reddit.com/r/algorithms/comments/13rcitf/confused_about_bigoh_theta_and_omega_notations/
8. <https://igotanoffer.com/blogs/tech/big-o-notation-and-complexity-analysis>
9. <https://www.programiz.com/dsa/asymptotic-notations>
10. <https://courses.cs.washington.edu/courses/cse373/19su/files/lectures/slides/lecture04.pdf>
11. https://www.reddit.com/r/computerscience/comments/s7h2vc/can_someone_explain_to_me_big_o_notation_like_im/
12. <https://www.sciencedirect.com/topics/computer-science/asymptotic-complexity>
13. <https://dev.to/coder7475/demystifying-asymptotic-analysis-time-space-complexity-ga3>
14. https://launchschool.com/books/advanced_dsa/read/time_and_space_complexity_recursive
15. <https://www.geeksforgeeks.org/dsa/what-is-the-difference-between-auxiliary-space-and-space-complexity/>
16. <https://www.geeksforgeeks.org/dsa/introduction-to-amortized-analysis/>
17. <https://www.enjoyalgorithms.com/blog/time-complexity-analysis-of-recursion-in-programming/>
18. <https://www.scaler.com/topics/data-structures/space-complexity-in-data-structure/>
19. <https://courses.cs.duke.edu/spring19/compsci330/lecture21scribe.pdf>
20. <https://yourbasic.org/algorithms/time-complexity-recursive-functions/>
21. <https://mudit.hashnode.dev/what-is-space-complexity-and-auxiliary-space>
22. https://en.wikipedia.org/wiki/Amortized_analysis
23. <https://builtin.com/software-engineering-perspectives/big-o-vs-big-theta>
24. <https://www.geeksforgeeks.org/dsa/how-to-analyse-complexity-of-recurrence-relation/>
25. <https://dev.to/mwong068/big-o-space-complexity-lcm>
26. <https://yourbasic.org/algorithms/amortized-time-complexity-analysis/>
27. <http://www.cs.ucf.edu/courses/cop3502/summer07/recursion12.pdf>
28. https://www.reddit.com/r/learnpython/comments/lbh5oz/whats_the_difference_between_auxiliary_space_and/
29. <https://www.cs.cmu.edu/~15451-f22/lectures/lec03-amortized.pdf>
30. <https://www.youtube.com/watch?v=EQTvYjYks08>
31. <https://www.geeksforgeeks.org/dsa/g-fact-86/>
32. <https://www.cse.cuhk.edu.hk/~taoyf/course/comp3506/lec/dyn-array.pdf>
33. <https://stackoverflow.com/questions/13467674/determining-complexity-for-recursive-functions-big-o-notation>
34. <https://www.geeksforgeeks.org/dsa/analysis-algorithms-big-o-analysis/>
35. https://en.wikipedia.org/wiki/Space_complexity
36. <https://www.geeksforgeeks.org/dsa/difference-between-big-oh-big-omega-and-big-theta/>

37. https://www.w3schools.com/dsa/dsa_timecomplexity_theory.php
38. https://en.wikipedia.org/wiki/Asymptotic_computational_complexity
39. <https://diffstudy.com/big-o-big-theta-and-big-omega-key-differences-explained/>